SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	M.SANA FATHIMA	**Reg. No.:**	192371082
Date:	25/08/2026	**Duration:**	60 minutes

Code of Conduct

I M. SANA FATHIMA_(192371082) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. SANA FATHIMA

Annexure A – Test Case Design Assignment

Problem 9 – E COMMERCE ORDER FULFILLMENT PLATFORM

Introduction

An E-Commerce Order Fulfillment Platform is a software system designed to manage the complete lifecycle of an online customer order, beginning with user registration and product selection and continuing through shopping cart management, checkout, payment, order confirmation, inventory update, warehouse processing, picking, packing, shipment, delivery, and order tracking. The platform connects different activities and users involved in the fulfillment process and therefore requires comprehensive testing to ensure that every stage works correctly and that information remains consistent between stages. The main purpose of testing this platform is to verify its correctness, functionality, reliability, usability, security, and ability to handle exceptional situations. Since an e-commerce platform deals with customer information, product inventory, payment transactions, and delivery details, even a small software defect can result in incorrect charges, duplicate orders, wrong products being shipped, incorrect stock information, delayed fulfillment, unauthorized access, or loss of customer trust. Therefore, the proposed test case design focuses on validating the major functional and non-functional requirements of the E-Commerce Order Fulfillment Platform and provides coverage for normal, boundary, invalid, and exceptional conditions. The assessment instructions specifically require identification of functional and non-functional requirements, test scenarios, detailed test cases, test data, expected results, and suitable test design techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing.

Objectives of Testing

The primary objective of testing the E-Commerce Order Fulfillment Platform is to establish confidence that the system performs all important business operations correctly and consistently. Testing should verify that a customer can create an account, log in securely, search for products, view product availability, add products to a shopping cart, modify cart quantities, remove products, provide valid delivery information, make a payment, and receive an order confirmation. Testing should also verify the fulfillment activities performed after an order is confirmed. A confirmed order should reach the appropriate processing stage, warehouse personnel should be able to identify the order, pick the correct products, pack them correctly, and prepare the order for shipment. The platform should update the order status as it moves through the fulfillment lifecycle and should allow the customer to track the order. Testing should additionally verify cancellation rules, out-of-stock handling, duplicate order prevention, payment failure handling, network interruption handling, session expiration, and unauthorized access. The overall objective is not only to test whether the system works under normal conditions but also to determine whether it behaves safely and predictably when invalid inputs, unexpected failures, or high workloads occur.

Functional Requirements

The E-Commerce Order Fulfillment Platform should satisfy several functional requirements. First, the platform should provide user registration so that new customers can create accounts using the required personal information. The registration process should validate mandatory fields and should prevent an account from being created when required information is missing or invalid. The platform should also provide a secure login facility through which registered customers can access their accounts using valid credentials. Incorrect credentials should be rejected and should not provide unauthorized access. Another important requirement is product search and selection. Customers should be able to search for products using names, keywords, or categories and should receive relevant product results. The system should show product availability so that customers can identify whether a product can be purchased. The shopping cart should allow customers to add available products, change quantities, and remove items. The cart should correctly display the selected products and calculate the applicable total. The platform should then support checkout and order placement by collecting valid delivery information and a selected payment method. Payment processing should correctly distinguish successful and unsuccessful transactions.

Non-Functional Requirements

In addition to functional requirements, the platform should satisfy important non-functional requirements. Performance is important because customers expect product searches, cart operations, checkout, and tracking requests to respond within an acceptable time. The platform should continue to operate effectively when a large number of users access the system simultaneously. Security is another major requirement because the system handles customer accounts, addresses, order details, and payment-related information. Unauthorized users should not be able to access protected information or perform restricted operations. Reliability is necessary because payment and order processing should not create inconsistent states when network interruptions or service failures occur.

Test Scenarios

The major test scenarios for the platform cover the complete order lifecycle. The first scenario is user registration, where valid customer information is entered and a new account is created. The second scenario is invalid registration, where mandatory information is missing or incorrect and the system should display appropriate validation messages. The third scenario is successful user login using valid credentials, followed by invalid login testing using an incorrect password or username. Product search should be tested with valid keywords as well as searches that return no matching product. Product availability should be verified for both available and out-of-stock products. Shopping cart operations should be tested by adding products, increasing and decreasing quantities, and removing products. Checkout should be tested using valid and invalid delivery information.

Detailed Test Case Design

Test Case 1 – Successful User Registration

The purpose of this test is to verify that a new customer can successfully register. The tester should open the registration page, enter a valid name, email address, mobile number, and password, and then submit the registration form. The test data should contain valid customer information. The expected result is that the system validates the information and creates the account successfully. A suitable confirmation message should be displayed and the new account should be available for login.

Test Case 2 – Registration With Missing Mandatory Information

The purpose of this test is to verify that the registration process does not accept incomplete information. The tester should open the registration page, leave a mandatory field such as the email address empty, enter valid information in the remaining fields, and select the registration option. The expected result is that the system should prevent account creation and should clearly identify the missing information. The system should not create a partially completed customer account.

Test Case 3 – Successful Login

The purpose of this test is to verify successful authentication. The tester should enter the registered email address and the correct password and then select Login. The expected result is that the system authenticates the customer and redirects the customer to the appropriate account or home page. The authenticated session should be created correctly.

Test Case 4 – Login With Incorrect Password

The tester should enter a registered email address together with an incorrect password and attempt to log in. The expected result is that the system should reject the login request, display an appropriate error message, and prevent access to protected customer information. No authenticated session should be created for the failed login attempt.

Test Case 5 – Product Search With Valid Keyword

The tester should open the product search facility, enter a valid product name or keyword such as Wireless Headphones, and submit the search. The expected result is that relevant products should be displayed. Product names and available information should correspond to the entered search criteria.

Test Case 6 – Product Search With No Matching Product

The tester should enter a product name or keyword for which no matching product exists and perform the search. The expected result is that the platform should display a clear message such as no products found rather than showing unrelated products. The system should handle the empty-result condition without crashing.

Test Case 7 – Add Available Product to Cart

The tester should search for an available product, open the product details, and select Add to Cart. The expected result is that the product should appear in the shopping cart with the correct product name, selected quantity, price, and applicable total. The cart should reflect the latest state immediately after the product is added.

Test Case 8 – Remove Product From Cart

The tester should add a product to the cart, open the cart, select the remove option, and verify the cart. The expected result is that the selected product should be removed and the cart total should be recalculated correctly. No removed product should remain as an active cart item.

Test Case 9 – Update Cart Quantity

The tester should add a product to the cart and increase its quantity. The expected result is that the new quantity should be displayed correctly and the total amount should be recalculated. The tester should also reduce the quantity and confirm that the system updates the cart correctly.

Test Case 10 – Valid Order Placement

The tester should add an available product to the cart, proceed to checkout, enter a valid delivery address, select a valid payment method, and confirm the order. The expected result is that the order should be created successfully and should proceed to the payment and fulfillment process.

Test Case 11 – Invalid Delivery Address

The tester should proceed to checkout and enter incomplete or invalid delivery information, such as an address without a required PIN code. The expected result is that the system should prevent the order from proceeding and should display an appropriate validation message. The invalid address should not be accepted as a confirmed delivery address.

Test Case 12 – Successful Payment

The tester should create a valid order and proceed to payment using valid payment information. The expected result is that the payment should be successfully processed and the order should move to confirmation. The system should record the successful payment state accurately.

Test Case 13 – Failed Payment

The tester should create a valid order and then provide invalid payment information or simulate a payment failure. The expected result is that the transaction should fail safely, the customer should receive a clear payment failure message, and the order should not incorrectly be marked as payment confirmed.

Test Case 14 – Order Confirmation

After a successful payment, the tester should verify the confirmation screen and order information. The expected result is that the platform should generate a unique order ID and display confirmation

details. The order should be stored with the correct customer, product, quantity, amount, payment, and delivery information.

Test Case 15 – Inventory Update

The tester should note the available quantity of a product, place an order for a known quantity, complete payment, and then check the product inventory. The expected result is that the available stock should decrease according to the purchased quantity. The system should not create an incorrect stock value.

Test Case 16 – Out-of-Stock Product

The tester should select a product whose available stock is zero and attempt to purchase it. The expected result is that the platform should prevent the purchase or clearly indicate that the product is unavailable. The system should not allow an out-of-stock product to be confirmed as an order.

Test Case 17 – Warehouse Receives Confirmed Order

The tester should place and successfully confirm an order and then access the fulfillment or warehouse module. The expected result is that the confirmed order should appear in the warehouse processing queue with the correct product and quantity information.

Test Case 18 – Product Picking

Warehouse personnel should open a confirmed order, identify the required products and quantities, pick the products, and confirm the picking activity. The expected result is that the correct products should be marked as picked and the order status should be updated appropriately.

Test Case 19 – Product Packing

The tester should select an order whose products have been correctly picked, verify the products, pack them, and confirm the packing stage. The expected result is that the order should be marked as packed and should become eligible for shipment.

Test Case 20 – Shipment Processing

The tester should open a packed order, assign the order to a delivery service, and confirm shipment. The expected result is that the order status should change to Shipped and the required shipment information should be stored correctly.

Test Case 21 – Order Tracking

The tester should log in to the customer account, open the orders section, select a valid order, and choose the tracking option. The expected result is that the system should display the current order status and available tracking information without exposing information belonging to another customer.

Boundary Value Analysis

Boundary Value Analysis should be applied to input values that have defined minimum and maximum limits. For example, if the platform permits a customer to purchase between 1 and 10 units of a particular product in a single order, the important test values are 0, 1, 2, 9, 10, and 11. A quantity of 1 represents the minimum valid value and should be accepted, while 10 represents the maximum valid value and should also be accepted. A quantity of 0 should be rejected because it does not represent a valid purchase quantity. A quantity of 11 should be rejected because it exceeds the permitted limit. Negative quantities such as -1 should also be rejected. Non-numeric values such as ABC should not be accepted when the quantity field requires a number. Testing values immediately at, below, and above the boundary helps identify validation defects that may not be detected by ordinary test cases.

Equivalence Partitioning

Equivalence Partitioning can be used to divide input values into groups that are expected to behave similarly. For product quantity, a valid partition may contain values from 1 through 10. Invalid partitions may contain zero, negative quantities, values greater than 10, alphabetic input, special characters, and empty input. Instead of testing every possible quantity, representative values from each partition can be selected. For example, a value such as 5 can represent the valid partition, -1 can represent the negative-value partition, 11 can represent the above-maximum partition, and ABC can represent the non-numeric partition. This technique reduces unnecessary test duplication while ensuring that the major classes of valid and invalid input are covered.

Decision Table Testing

Decision Table Testing is useful when the outcome of an operation depends on a combination of conditions. In the order placement process, three important conditions are product availability,

delivery address validity, and payment success. When the product is available, the delivery address is valid, and the payment is successful, the order should be confirmed. If the product is unavailable, the system should prevent the purchase and display an out-of-stock message. If the address is invalid, the customer should be asked to correct the delivery information. If payment fails, the system should not incorrectly confirm the order. Testing these combinations ensures that the platform handles different business conditions correctly and does not produce an incorrect result when one or more conditions change.

State Transition Testing

State Transition Testing is especially important for an order fulfillment platform because an order changes from one state to another throughout its lifecycle. A normal order can move from Order Placed to Payment Confirmed, then to Processing, Picked, Packed, Shipped, Out for Delivery, and finally Delivered. Alternative states can include Payment Failed and Cancelled. The system should allow only valid transitions. For example, an order that has been successfully placed and paid can move into processing, while a failed payment should move into a payment failure state rather than payment confirmation. An order may be cancelled while cancellation is allowed, but a shipped or delivered order should not be moved backward into an earlier fulfillment stage unless a specific business rule permits it. Testing these transitions verifies that the order state remains logically consistent.

Exception and Negative Testing

Exception and negative testing are necessary because real-world e-commerce transactions can encounter unexpected conditions. The platform should be tested when the internet connection is interrupted during payment, when the payment gateway becomes unavailable, when the user repeatedly clicks the Place Order button, when a product becomes unavailable during checkout, when the warehouse service becomes temporarily unavailable, when shipment information cannot be retrieved, when a customer session expires during checkout, when an invalid order ID is entered, and when an unauthorized user attempts to access another customer's order. The expected behavior in these cases is graceful handling without data loss, incorrect order confirmation, duplicate payment, unauthorized disclosure, or inconsistent inventory. The system should provide clear messages and preserve a safe transaction state.

Security Testing

Security testing should verify that customer and order information is protected from unauthorized access. Invalid users should not be allowed to enter protected accounts. Customers should only be able to view their own order information, and warehouse or administrative functions should be available only to authorized personnel. Session expiration should prevent continued access after a session becomes invalid. Manipulating an order ID should not provide access to another customer's order. The system should also prevent unauthorized modification of order status and should protect sensitive customer and payment-related information. These tests are important because the assessment emphasizes security, reliability, and ethical quality assurance practices.

Performance Testing

Performance testing should verify whether the platform can continue to operate correctly when many customers use it at the same time. The system should be tested with simultaneous product searches, cart operations, checkout requests, payment transactions, warehouse updates, and order tracking requests. The purpose is to determine whether the system remains responsive and whether increased workload causes crashes, incorrect transactions, duplicate orders, or data corruption. Performance testing should be carried out under normal workload as well as increased workload so that the behavior of the platform can be evaluated under different operating conditions.

Usability Testing

Usability testing should verify whether customers can understand and complete the major activities of the platform without unnecessary difficulty. Registration and login screens should be clear, product search should be easy to locate, product details should be understandable, and the Add to Cart and Checkout actions should be clearly presented. The cart should clearly show selected products, quantities, and totals. Checkout should guide the customer through delivery and payment steps. Error messages should explain what needs to be corrected. Order confirmation should clearly show the order ID, and order tracking should be easy to locate. A usable platform reduces customer confusion and improves the overall experience.

Reliability and Data Integrity Testing

Reliability testing should verify that the platform maintains correct information even when unexpected failures occur. Network interruptions, payment timeouts, temporary service failures, repeated requests, and session expiration should be tested. The system should avoid creating duplicate orders and should not confirm a transaction unless the payment state is known to be successful. Data integrity testing should verify that customer information, product information, inventory, payment status, order status, and fulfillment status remain consistent. For example, after an order is successfully confirmed, the inventory should reflect the purchased quantity, the order should appear in the fulfillment system, and the payment status should correspond to the actual transaction result. Any mismatch between these connected components can lead to serious business problems and should therefore be treated as a defect.

Test Data

The test data for the platform should include valid, invalid, boundary, and exceptional data. Valid data should include properly formatted customer names, valid email addresses, registered credentials, available products, valid delivery addresses, valid quantities, valid payment information, and existing order IDs. Invalid data should include incorrect passwords, invalid email addresses, incomplete delivery addresses, negative quantities, zero quantities, values above the allowed quantity, non-numeric input, invalid payment information, and non-existing order IDs. Boundary data should include values at the minimum and maximum permitted limits and values immediately outside those limits. Exceptional data should include network interruption, payment timeout, service unavailability, expired sessions, repeated order requests, and unavailable products. Using different categories of test data helps ensure that the platform is tested comprehensively rather than only under ideal conditions.

Expected Order Fulfillment Flow

The expected order fulfillment flow begins when the customer logs into the platform and searches for a required product. The customer selects an available product and adds it to the shopping cart. After reviewing the cart, the customer proceeds to checkout and enters delivery information. The system validates the address and other required information. The customer then selects a payment method and completes the payment. When payment is successful, the system confirms the order and generates a unique order ID. Inventory is updated and the confirmed order is transferred to the fulfillment process. Warehouse personnel process the order, identify the required products, pick the

correct quantities, and pack them. The packed order is then prepared for shipment and assigned to the appropriate delivery service. The order status is updated to Shipped and the customer can track the shipment. The order continues through the delivery process until it reaches the customer and is marked Delivered. At every stage, the system should preserve accurate and consistent information and should prevent invalid state transitions.

Requirement Traceability and Test Coverage

The proposed test cases provide coverage across the major requirements of the E-Commerce Order Fulfillment Platform. User registration and authentication are covered by the registration and login test cases. Product search and product availability are covered by the search and stock-related tests. Shopping cart functionality is covered through adding, removing, and updating products. Checkout and order placement are covered through valid and invalid delivery information tests. Payment processing is covered through successful and failed payment scenarios. Order confirmation and inventory management are covered by confirmation and inventory tests. Warehouse fulfillment is covered through order receipt, picking, packing, and shipment tests. Order tracking and cancellation are covered through valid and invalid tracking and cancellation scenarios. Duplicate order prevention and network failure handling provide exceptional-condition coverage.

Figure 1 – Customer Login Screen

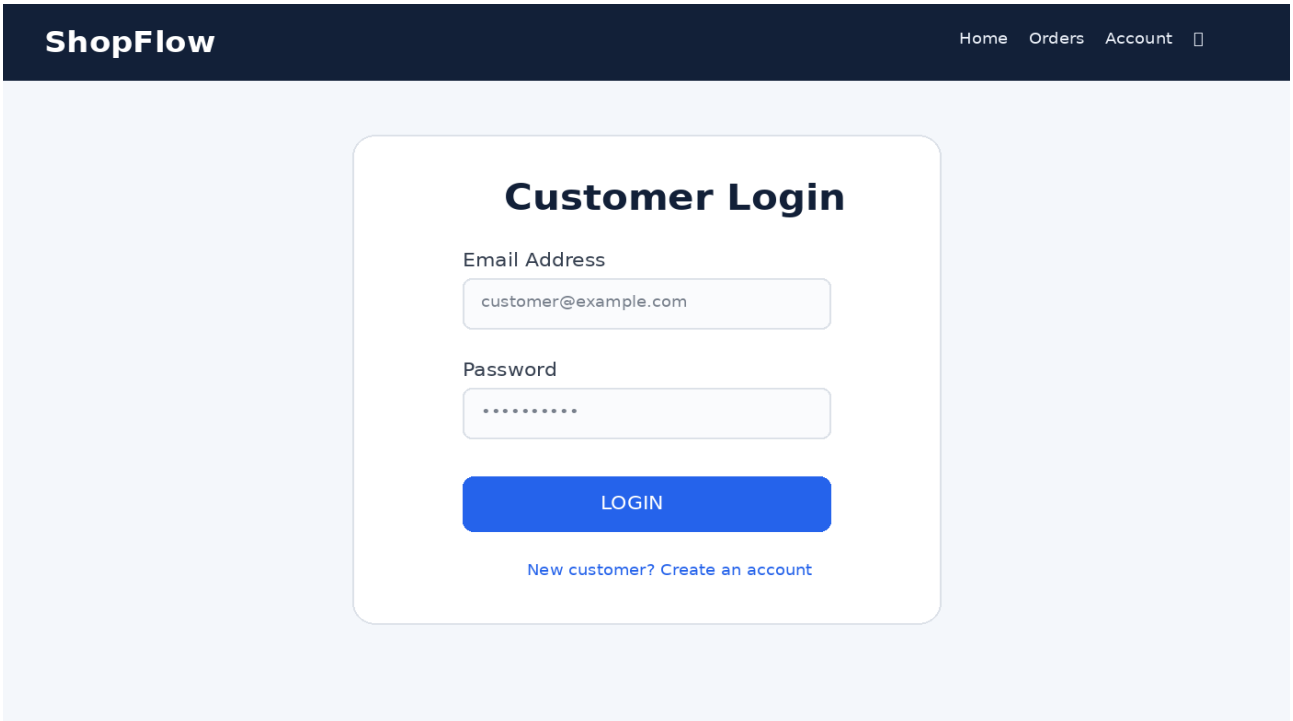


Figure 2 – Product Search and Selection Screen

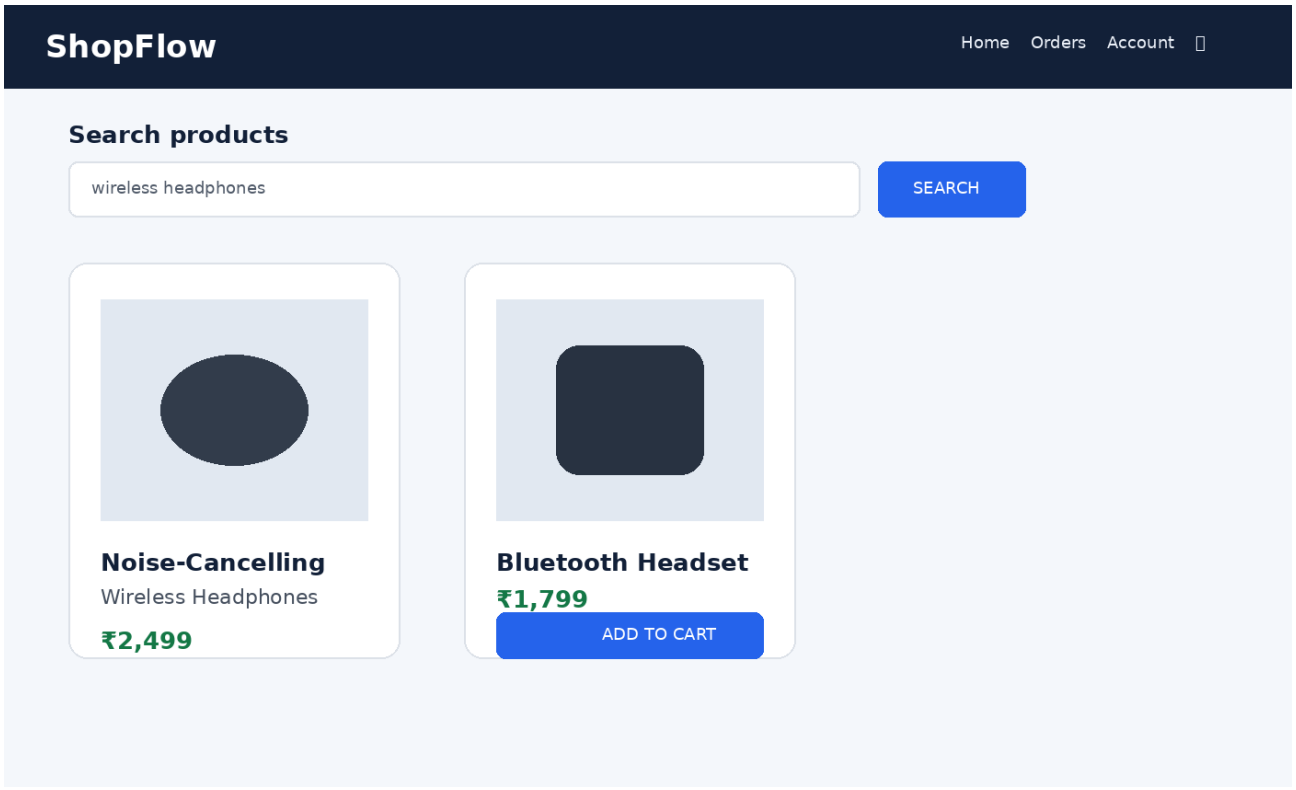


Figure 3 – Shopping Cart Screen

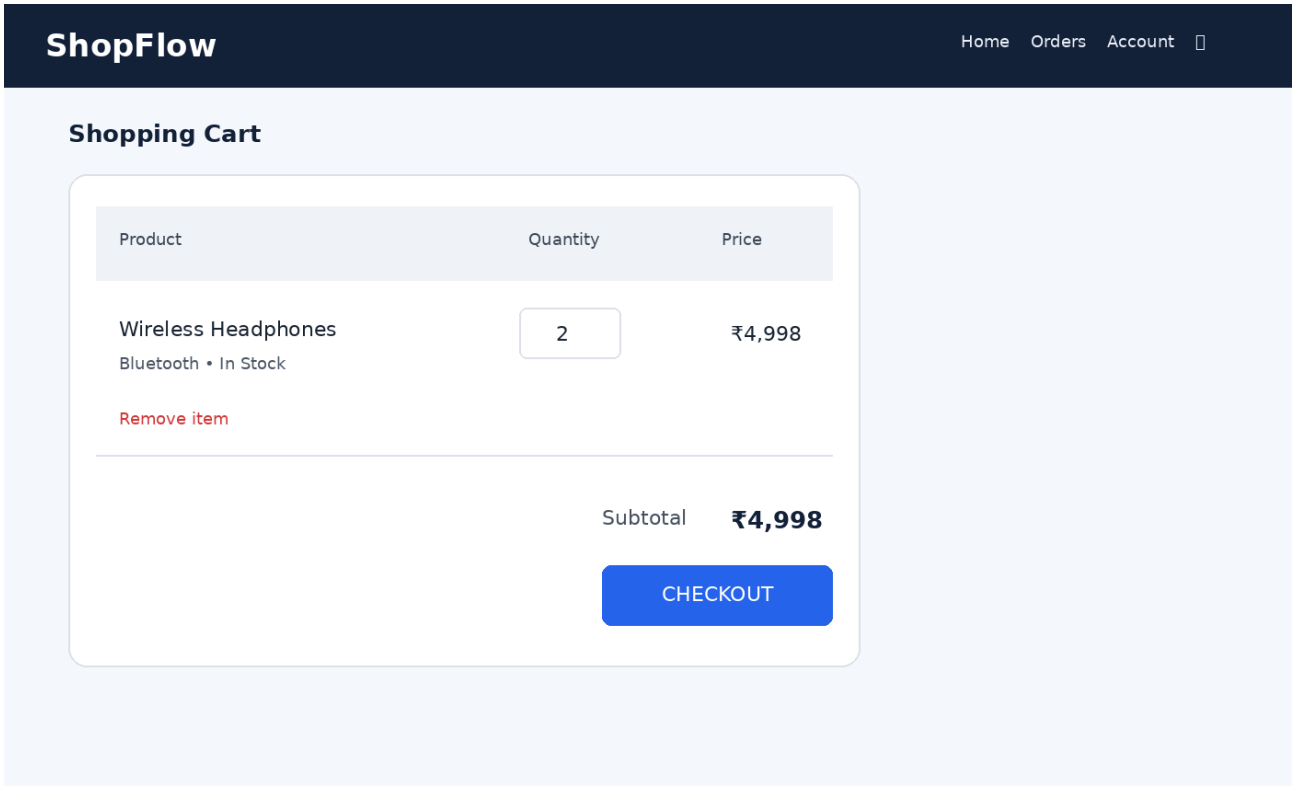


Figure 4 – Checkout and Payment Screen

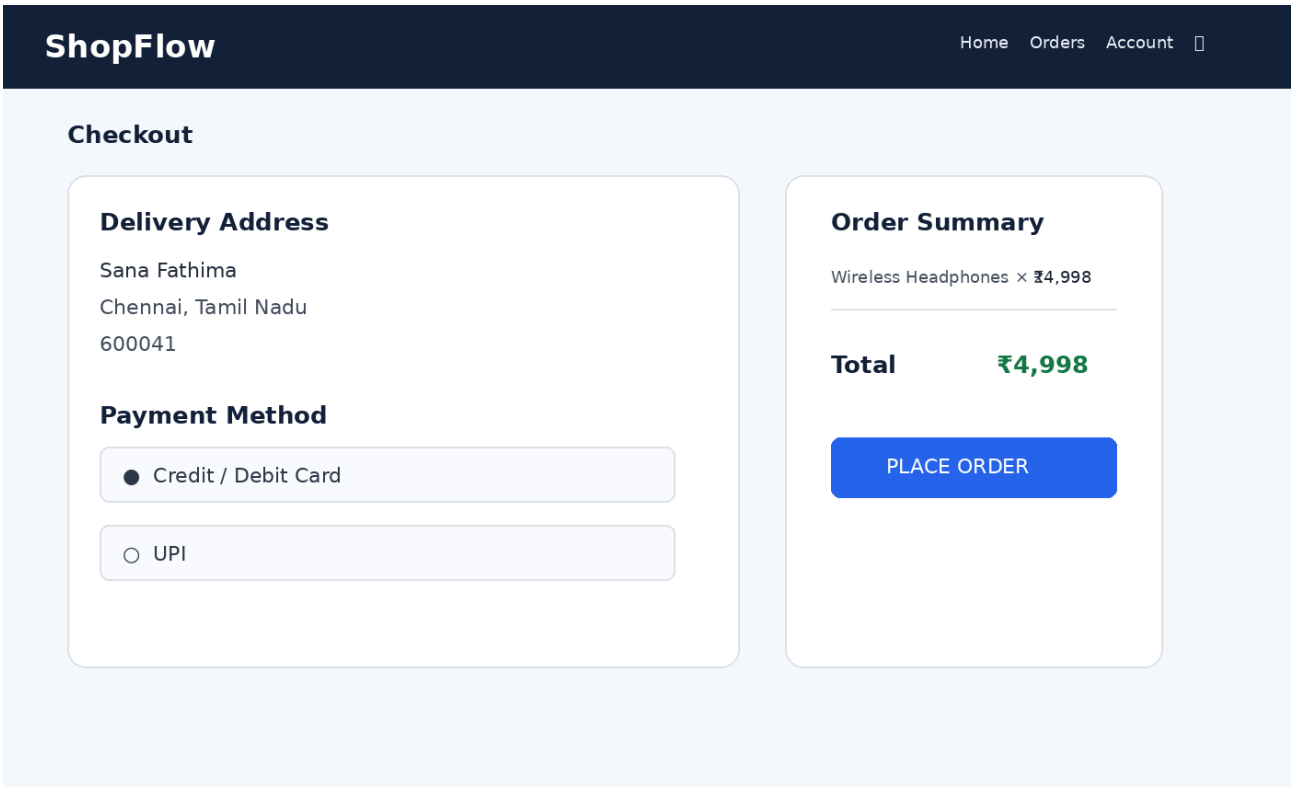


Figure 5 – Order Confirmation Screen

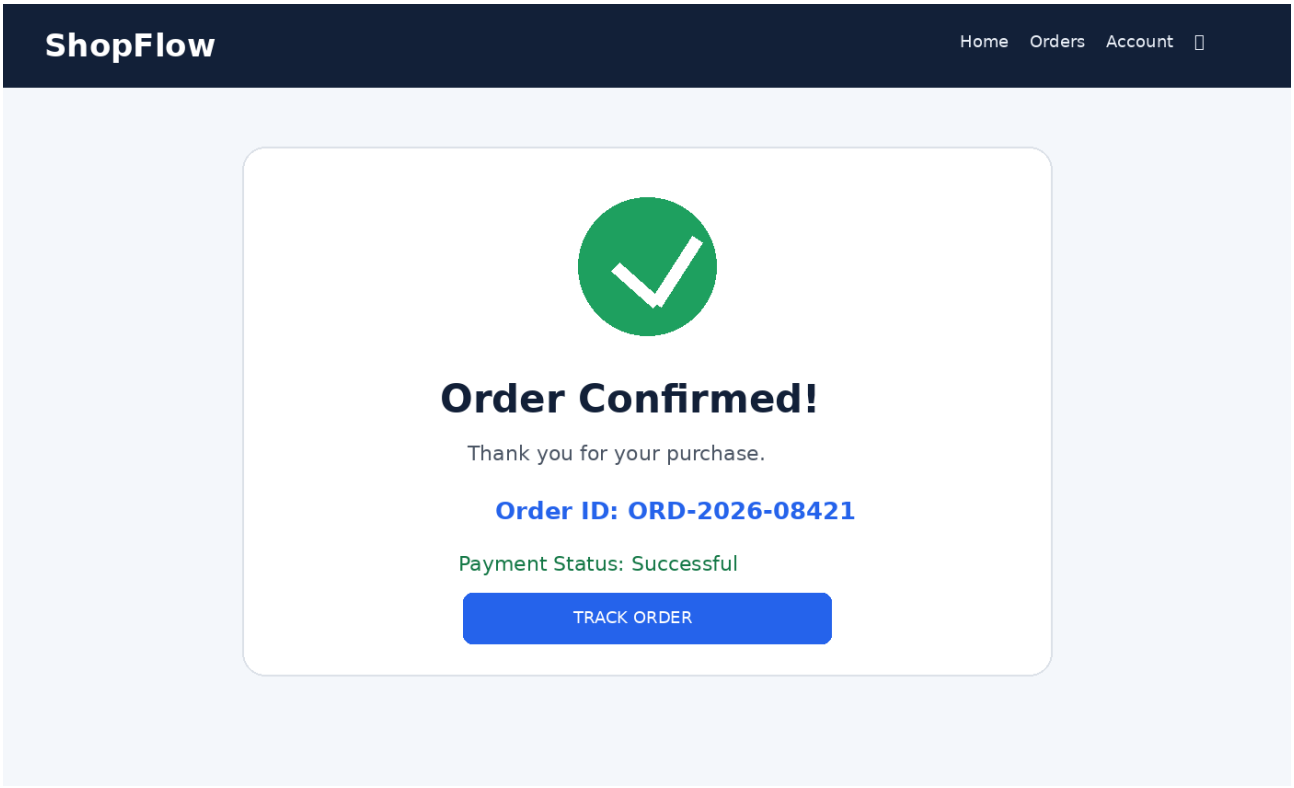
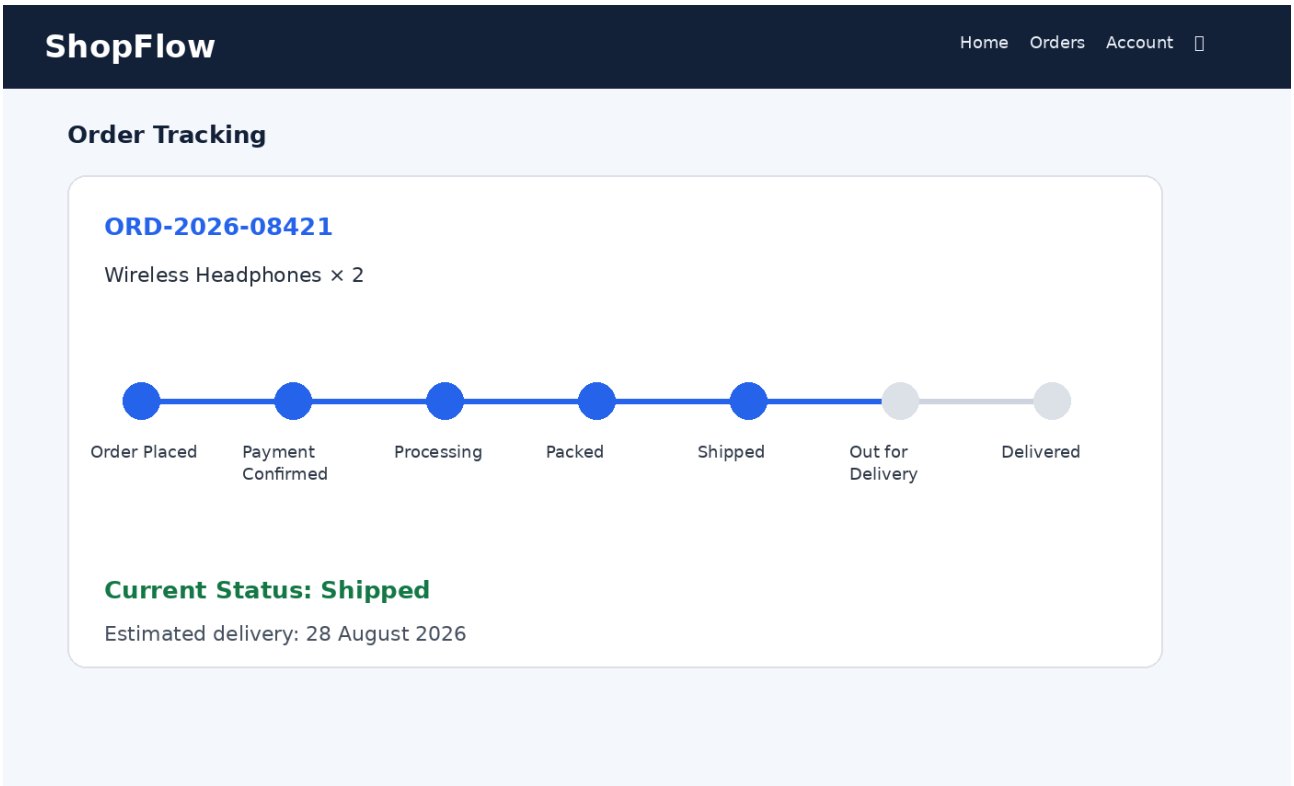


Figure 6 – Order Tracking Screen



Conclusion

The E-Commerce Order Fulfillment Platform is a complex system because it connects customer activities, product information, inventory, payment, warehouse operations, shipment, and delivery tracking into a single order lifecycle. Comprehensive testing is therefore necessary to ensure that every part of the process behaves correctly and that information remains accurate between different stages. The proposed test case design covers normal conditions, invalid inputs, boundary conditions, exceptional situations, security concerns, performance requirement.